Résolution parallélisée de Sudoku

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

FIGURE 1 – Une grille de sudoku incomplète

Le sudoku est un jeu de réflexion où l'objectif est de compléter une grille en plaçant un chiffre de 1 à 9 dans chaque case tout en satisfaisant les contraintes suivantes :
- chaque chiffre ne peut apparaître qu'une fois par ligne
- chaque chiffre ne peut apparaître qu'une fois par colonne
- chaque chiffre ne peut apparaître qu'une fois par bloc (groupement carré de 9 cases)

Vérification d'une grille

Dans l'archive fournie, il y a quelques grilles (`sudoku1.sdk` à `sudoku5.sdk` , par ordre croissant de difficulté) et un fichier `sdk.c` contenant le code à compléter. Nous représenterons une grille en mémoire par un tableau à deux dimensions contenant des nombres de 1 à 9 (s'il y a un nombre inscrit dans la case) ou 0 (pour symboliser une case vide).

Question 1 *Complétez la fonction* `readGrid` *qui complète une grille passée en paramètre en lisant un fichier texte dont le chemin est également passé en paramètre. Vous pouvez utiliser la fonction* `fscanf` *pour traiter les lignes du fichier.*

Question 2 *Complétez les trois fonctions* `checkColumn`, `checkRow` *et* `checkBlock` *qui prennent en paramètre un pointeur vers la grille et un numéro entre 0 et 8 pour identifier la ligne/colonne/bloc à vérifier. Ces fonctions renvoient un booléen qui indique si cette grille respecte les contraintes du sudoku sur la ligne/colonne/bloc en question. Complétez la fonction* `checkGrid` *qui vérifie si une grille satisfait toutes les contraintes du sudoku à l'aide des fonctions précédentes. Des grilles invalides sont fournies dans l'archive.*

On va chercher à optimiser ce code : on va créer trois threads. Le premier va vérifier les lignes d'un sudoku, le second les colonnes, et le dernier vérifiera les blocs.

Question 3 *Complétez les fonctions* `checkAllColumns`, `checkAllRows` *et* `checkAllBlocks` *qui prennent en paramètre un pointeur vers la grille et qui renvoient un booléen indiquant si la grille satisfait les contraintes associées (* `checkAllColumns` *vérifie que toutes les colonnes satisfont les contraintes, etc.). Complétez ensuite la fonction* `checkGridParallel` *qui vérifie si une grille satisfait toutes les contraintes du sudoku en appelant les fonctions* `checkAllColumns`, `checkAllRows` *et* `checkAllBlocks` *dans trois threads.*

Résolution d'un sudoku

Pour résoudre un sudoku, nous allons utiliser l'algorithme nommé *backtrack* qui est un algorithme de *bruteforce*. Son idée est simple : remplir les cases une par une. Si on obtient une grille qui ne satisfait pas les contraintes, on revient en arrière (si par exemple la dernière action avait été de mettre un 7 dans un case, on le retire et on met un 8 à la place). L'algorithme peut être écrit en pseudocode de la manière suivante :

```
Function backtrack(g : char[][]) : bool is
    if g n'a aucune case vide then
        if g' respecte les contraintes du Sudoku then  output g return True ;
        else  return False ;
    end
    row, column ← indice de la première case non-vide dans g
    for n de 1 à 9 do
        g' ← une copie (profonde) de g
        g'[row][column] ← n
        if g' respecte les contraintes du Sudoku then
            res ← backtrack(g')
            if res est True then  return True ;
        end
    end
    return False
end
```

Dans cet algorithme, la copie profonde de g n'est pas obligatoire mais sera utile par la suite.

Question 4 *Complétez la fonction* `backtrack` *qui implémente cet algorithme. Pour vérifier qu'elle fonctionne bien, faîtes le tourner sur la grille sudoku1.sdk qui peut être résolu en moins de 100 itérations. Vous pouvez également utiliser la fonction* `print_grid` *(déjà fournie) pour afficher la grille à chaque itération.*

Question 5 *Modifier cette implémentation pour la rendre parallèle. Copier* `backtrack` *en* `backtrackParallel` *et remplacez les appels récursifs dans la boucle* `for` *par la création de nouveaux threads.*

Question 6 *Évaluez les performances des deux algorithmes de résolution en termes de nombre d'itérations et de temps d'exécution. Que pouvez-vous en conclure ?*